Assignment 2 - Theory: Arithmetic of CNNs

For a convolution with dilation 1, the output size on each spatial axis is:

O = floor((N + 2P - K) / S) + 1

N is the input size, P the padding on each side, K the kernel size, and S the stride. The output channel count equals the number of filters. Shapes below use height x width x channels unless BCHW is explicitly stated.

Question 1

Input: 32 x 32 x 3. Eight 5 x 5 filters, stride 1, no padding.

H_out = W_out = floor((32 + 2(0) - 5) / 1) + 1
= floor(27) + 1 = 28.

There are 8 filters, so the output is 28 x 28 x 8. Each filter spans all 3 input channels and produces one output channel. In PyTorch batch notation: (B, 8, 28, 28).

Question 2

Keeping Question 1's settings except using same padding, stride 1 preserves spatial size. For a 5 x 5 kernel:

P = (K - 1) / 2 = (5 - 1) / 2 = 2 pixels on each side.

H_out = W_out = floor((32 + 2(2) - 5) / 1) + 1
= 31 + 1 = 32.

The output changes from 28 x 28 x 8 to 32 x 32 x 8.

Question 3

Input: 64 x 64. Kernel: 3 x 3. Stride: 2. Padding: 0.

H_out = W_out = floor((64 + 2(0) - 3) / 2) + 1
= floor(61 / 2) + 1
= floor(30.5) + 1 = 31.

The output spatial size is 31 x 31. The floor is necessary: an incomplete window is not counted.

Question 4

Input feature map: 16 x 16. Max pooling: 2 x 2, stride 2. Assume no padding and the usual floor mode.

H_out = W_out = floor((16 - 2) / 2) + 1
= 7 + 1 = 8.

The output is 8 x 8 per channel. Max pooling preserves the channel count.

Question 5

Input spatial shape: 128 x 128. Each convolution has kernel 3 x 3, stride 1, same padding.

P = (3 - 1) / 2 = 1.
First layer: floor((128 + 2(1) - 3) / 1) + 1 = 127 + 1 = 128.
Second layer: floor((128 + 2(1) - 3) / 1) + 1 = 128.

The final spatial shape is 128 x 128. The final channel count cannot be determined because the number of filters is not specified; it equals the second layer's filter count.

Question 6

model.train() switches a model and its child modules to training mode. It does not itself perform training or enable gradients.

Removing this line does not always change the result:

- A newly constructed PyTorch model defaults to training mode. If model.eval() has never been called, removing model.train() initially makes no difference to mode.
- If evaluation previously called model.eval(), removing model.train() leaves the model in evaluation mode during the next training loop.
- In the class notebook's EnhancedCNN, Dropout randomly drops activations in training mode but is disabled in evaluation mode. BatchNorm uses batch statistics and updates running statistics in training mode; in evaluation mode, its default behavior uses stored running statistics without updating them. Leaving this model in evaluation mode therefore changes training behavior and can harm results.
- The exact CNN required in this assignment contains Conv2d, ReLU, MaxPool2d, Flatten, and Linear, with no Dropout or BatchNorm. These layers behave identically in train/eval modes, so removing this line alone does not change its forward computations.

Gradients can still be computed in evaluation mode, and loss.backward() and optimizer.step() can still update parameters. torch.no_grad() and torch.inference_mode() are separate gradient-control mechanisms. Explicitly calling model.train() remains good practice before training and after validation.